Zenith: Dated Evaluation and Controlled Updates of a Reasoning Graph

Implementation and retrospective evaluation

Will Cook*

Draft, 11 October 2026

Abstract

Zenith evaluates changes to a reasoning graph using dated inputs, native output contracts and complete paired forecast support. Its executor binds runs to source, graph, policy and provider identities; diagnostics distinguish recorded failures from hypotheses about their cause. A proposal compiler can change one node-policy field for controlled replay. A historical price-only study compared one manually frozen persistence/scale rule with a model baseline over 20 August and five later October 2015 dates. On the later dates, candidate mean relative price error was 0.8875%, versus 0.9966% for the model baseline and 1.0453% for no change. The 100-call study demonstrates pipeline feasibility and a descriptive intervention comparison. Previously examined history, a modern model and limited support prevent claims of prospective accuracy, learned improvement or novelty. The next question is whether a rule derived from measured development feedback improves later forecasts over the frozen manual rule under matched execution conditions.

1 Question and scope

A poor forecast can reflect unavailable evidence, restricted context, an unsupported inference, model-response variation or an unpredictable outcome. Final price error does not identify which explanation applies. A useful learning loop must turn measured failures into a declared intervention, produce new forecasts through the same executor and assess them on later matched evidence.

Zenith is a standalone extraction of an existing Lab–Oracle–Refinement system. Lab emits a thesis checkpoint (CP1) and forecast checkpoint (CP2); Oracle resolves later observations. The authored inventory has 71 nodes, with 28 in the default Lab plan. These counts describe structure, not successful execution. The retained refinement runner and the newer reflection/cohort route are separate implementations. The completed experiment uses an explicitly admitted three-node price-only replacement: a stock feed, CP1 and CP2. It does not test the full Lab architecture.

Graph-based reasoning is established prior art [2]. The question here is narrower: can measured development feedback produce a bounded node policy that improves later forecast loss over a frozen manual rule? The completed study tests a manual rule, so it establishes the execution and comparison boundary but does not answer that learning question.

*AI agents researched and drafted this manuscript under Will Cook's direction. The source and experimental receipts define the evidence reported here.

2 Implemented mechanism and its guarantees

2.1 Native execution and evidence boundaries

A node declares dependencies, instructions and an output schema. A reasoning policy can add a short skill rule, set nonnegative incoming-edge weights or limit context characters. Base instructions and output contracts remain binding. Canonical artifacts are retained; context views contain full artifacts, declared excerpts or omission markers. Receipts record source hashes, retained characters and missing dependencies.

Declared edges are scheduled inputs. Effective requests also contain injected contracts, temporal substitutions and configured context. Agent-provider handoff packets retain complete prompts; the paid adapter records prompt hashes, and engine attempt sidecars retain response text with at most 4,000 prompt characters. A graph path therefore directs inspection without proving actual model consumption or causal responsibility. Native output validation applies to both paid responses and imported agent responses. Paid cohorts require the same requested provider endpoint in both arms and check returned provider identity against its catalog binding.

Observation time, first availability and acquisition time are distinct. Forward evidence must belong to the subject-side class and have declared availability no later than the issue. Unknown availability remains uncertain. Conservative historical release rules do not authenticate publication clocks or price vintages. Later truth is separated from forward execution.

The conditional Lean model proves composition when the constituent skills satisfy their assumed relations, and expresses availability and rank-based acyclicity constraints. The Python checker verifies declared ports, evidence classes, source hashes and finite ranks. Neither establishes that a natural-language skill implements its relation, that a timestamp is authentic, or that the forecast is accurate.

2.2 Measurement and proposals

Each forecast names its target, horizon, issue and target prices, basis and currency. Before maturity it is pending; missing or incompatible truth is a data gap. For issue i, fixed target set S_i, predictions q_{is} and positive realized prices y_{is}, the cohort loss is

$$L_i = \frac{1}{|S_i|} \sum_{s \in S_i} \frac{|q_{is} - y_{is}|}{y_{is}}, \qquad L = \frac{1}{n} \sum_{i=1}^{n} L_i.$$

Targets are averaged within issues, then issues receive equal weight. Complete comparison requires every included arm to have the exact frozen issue/target support. Partial runs expose coverage without aggregate, per-issue or paired scores. Complete receipts retain $L_i^{\text{candidate}} - L_i^{\text{baseline}}$; negative values favor the candidate.

This is a point-price loss, not a trading return. No-change and past-one-session momentum use the same support and loss. For positive outcomes with finite inverse moment, absolute percentage error elicits an inverse-price-weighted median, generally distinct from a conditional mean [6]. Current instructions ask for a target price without naming that functional. Any subsequent change to the elicited quantity or loss must be declared before execution, rather than rescoring the existing study opportunistically.

Diagnostics bind artifact presence, validator rejection and context-delivery conditions to recorded bytes and receipts. Missing or unbound evidence stays unknown; observer failure cannot change cohort acceptance. A large final error can select a trace for inspection but cannot certify missing data, data-access failure or faulty reasoning as its cause.

The reflection route binds development membership, feedback and selected node artifacts plus available agent-provider exchanges. It does not consume all paid debug sidecars; without a handoff packet the selected input prompt is unavailable. An agent supplies a hypothesis and intervention. The compiler checks the measured baseline, changes one skill rule, context budget or edge weight through the policy validator, and records before/after identities. Compilation neither executes nor accepts the candidate. An input intervention needs a separately declared acquisition/ablation protocol. Coupled changes can be tested through baseline/A/B/A+B replay; single-field changes have no general optimality guarantee.

3 Completed historical experiment

3.1 Fixed protocol and provenance

The experiment uses StockNet price evidence [9], with AAPL, MSFT and GOOG fixed throughout. It supplies 20 past exchange sessions of OHLCV and predicts the next exchange-session close. Development contains the first 20 common mature August 2015 issue dates; later assessment contains the first five October dates. The native cohort freezes candidates, inputs and source identities, separates split membership by calendar week, checks the declared one-day embargo, and seals final assessment before opening its truth.

Both arms use the same three-node graph, model, provider and 24,000-character context caps. Their sole difference is one CP2 skill rule: use current close as a persistence prior, separate direction from move size, and shrink movement toward zero when past-return scale evidence is weak, while preserving target and grounding requirements. This rule was manually fixed before development scoring, not generated by measured feedback. The baseline and candidate policy identities are frozen in the cohort at prepared source revision `01522cc`, hash prefix `bd5224387591`; full identities are in `paper/study-evidence.json`.

The August outcomes had been exposed in an earlier flat-input study, and the October dates had been examined in deterministic benchmark work. This freeze supplies a controlled historical comparison, not a pristine holdout. The modern model may also encode historical outcomes. Valid input timestamps cannot remove pretrained look-ahead knowledge; anonymization controls can test some effects without proving absence of memorization [5].

Four compatibility pilots used 1 July 2015, excluded from both study windows. They retain native failures rather than converting format, length or transport errors into reasoning diagnoses (Table 1). Prompt profile v2 increased requested word margins while preserving the validator. Its introduction at `7582bf1` is distinct from executed implementation content: all 105 recorded implementation hashes match `c4c051c` for Nova

and Qwen, `aa2f8ec` for failed DeepSeek, and `01522cc` for the accepted DeepInfra pilot. The manifests do not record Git HEAD; these matches establish content compatibility, not an observed or unique executed revision.

Table 1: Excluded compatibility pilots. Native acceptance does not measure forecast accuracy. Charges are provider-reported USD.

Model/route	Attempts	Native outcome	Known charge
Nova Micro	1	CP1 rejected: 36 words, minimum 150; invalid references.	0.000307895
Qwen Flash	2	CP1 accepted; CP2 rejected: 145 words, minimum 165.	0.001489930
DeepSeek	1	HTTP 429; no response or usage; CP2 unrun.	Unknown
DeepSeek/DeepInfra	2	CP1 and CP2 accepted.	0.002375240

The HTTP 429 retained an unknown charge and its full reservation. Explicit hash-bound recovery preserved the failed record and budget limits; a fresh pinned pilot replaced no existing attempt. The four pilots total six attempts, five responses and USD 0.004173065 in known charges. Their exact model, validation, token and hash evidence is retained in `paper/calibration-evidence.json`.

3.2 Complete common-support results

All study requests named `deepseek/deepseek-v4.1-flash`, requested endpoint `deepinfra/fp8`, and returned the matching model through reported DeepInfra. Development completed 40 episodes across 20 dates and four calendar weeks; later assessment completed 10 episodes across five dates and two weeks. Each arm has three scored targets per date. No study calls failed or had unknown cost. The later receipt matches the state sealed once under `native-deepinfra-final-v1`.

Table 2: Complete study loss as percentages, $100L$. Each issue has three fixed targets; deterministic references share support but are not model arms.

Phase	Dates	Baseline	Candidate	No change	Momentum
Development	20	2.1000	1.9142	1.8254	2.3756
Later assessment	5	0.9966	0.8875	1.0453	0.9038

Development candidate-minus-baseline mean issue loss was -0.00185743, lower on 14 dates and higher on six. The candidate remained worse than no change, but passed the frozen rule for a later candidate proposal. Later mean paired loss was -0.00109107, lower on three dates and higher on two; the candidate was below all three reference forecasts on this support. The native final decision remained `insufficient_issues`: five dates are below its 20-issue proposal gate, and the receipt permits no live mutation. These routing thresholds are not significance tests.

Development used 80 responses at USD 0.0992369112; later assessment used 20 at USD 0.0244918688. The complete study cost USD 0.1237287800. With calibration, the settled ledger has 106 attempts, 105 responses and USD 0.1279018450 in known charges, plus the earlier unknown-cost HTTP 429. It retains USD 1.137895220 in reservations and is not halted; reservations are not actual charges. Exact scores, paired losses, target coverage and accounting remain in the immutable study export. This small retrospective comparison has no unchanged-baseline replicates and estimates no uncertainty, profitability or prospective forecasting advantage.

4 From a manual intervention to measured learning

4.1 Closest prior art and originality boundary

QuantAgent's 2024 writer/judge loop retrieves experience, while an outer market-data evaluation retains signal code, scores and feedback [7]. Its alpha-factor information-coefficient/Sharpe task is different from point-price forecasting. FinCon compares training trajectories and selectively propagates revised beliefs to fixed financial-agent roles [10]. These are close prior art for the entire feedback loop and for using outcomes to revise particular instructions. Their return and portfolio results are not comparable to the loss in Table 2.

Trace propagates executed subgraphs and output feedback to trainable parameters; OptoPrime proposes prompt, code or numerical updates [4]. GEPA reflects on execution feedback and searches prompt candidates [1]. Zenith presently compiles a reviewed node-policy intervention for native replay. It does not implement either optimizer or unrestricted generated programs. A skill change on a fixed graph is policy adaptation, not learned topology. Dated admission, diagnostics and frozen comparison are the local implementation focus; no new learning algorithm or established novelty is claimed. Reviewed versions, software pins and broader comparisons remain in the source manifests, rather than forming a citation inventory here.

4.2 Controlled routing validation

A separate deterministic benchmark composes the real context-policy owner with a Decimal split-adjusted-price calculation and an independently implemented Fraction oracle, without model calls. Across seven scenarios, a finite selector scores unchanged policy and one-edge restorations witnessed in omission receipts on 20 training cases; unchanged wins ties. The selected policy is fixed before 64 assessment cases with disjoint price/factor ranges are generated. Execution accepts only sources and cutoff, rejects oracle labels, validates policy hashes and preserves canonical inputs. Restoring the relevant split edge yields 64/64 contract-correct numeric answers (absolute tolerance 10^{-25}), versus 0/64 for unchanged or irrelevant-edge restoration. Absent, future and conflicting sources yield 64 correct abstentions and zero numeric answers each; healthy and irrelevant-omission controls retain unchanged policy and 64 correct answers. A faulty arithmetic operator stays wrong in all 64 cases under every routing arm. Restoration increases delivered context, so this establishes no fixed-budget efficiency. Published seeds, constructed faults and a restricted candidate set make this integration/control

validation, not an LLM skill-learning result, causal discovery or financial forecasting evidence. The source-bound receipt is `docs/repair-controls.json`.

5 Limits and reproduction

The completed result establishes native execution and a bounded historical intervention comparison. It does not establish automatic credit assignment, causal diagnosis, generalization of a learning procedure, usefulness of the full Lab graph, market efficiency or profitability. Context restrictions, missing evidence and reasoning can interact; an intervention measures conditional sensitivity without uniquely identifying the original cause. A modern model's stored knowledge, previously inspected windows, serially related dates and three selected assets limit broader interpretation. Generation and selection are fitting [3]; a later learning study must freeze proposals before outer assessment and disclose lifetime exposure. An unchanged-rule repeat estimates model variation; a feedback-free proposal control distinguishes feedback value from making another proposal. Neither is present here. Repeated search also needs dependence-aware uncertainty over its full comparison family, for which White's Reality Check is one framework [8], not a statistic implemented or estimated in this study.

The repository contains executor/schema source and text-free evidence projections, not redistributed market text or private provider packets. Candidate code is trusted local code; the runner is not a security sandbox. Client-side reservations and reported usage do not guarantee provider billing.

The source-owner map is in `paper/README.md`; `docs/reproducing.md` owns environment and offline checks. Source and finance manifests retain primary versions, software pins and adoption decisions.

From an editable source checkout, `python -m zenith validate` and `python -m zenith plan lab` inspect the authored graph without inference. The paper exporter reads existing receipts without evaluating, selecting or sealing. Its validation accepted the actual complete phase reports and rejected copied missing-support, altered-loss and invalid-selection probes while preserving the original artifacts. Every measured claim must bind source, inputs, graph/policy and model/provider receipts; a code citation alone cannot reproduce the comparison.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. arXiv:2507.19457v2, 2025. URL https://arxiv.org/abs/2507.19457v2. Version read: 2507.19457v2.

[2] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690,

2024. doi: 10.1609/aaai.v38i16.29720. URL https://ojs.aaai.org/index.php/AAAI/article/view/29720.

[3] Gavin C. Cawley and Nicola L. C. Talbot. On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010. URL https://www.jmlr.org/papers/v11/cawley10a.html.

[4] Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the Next AutoDiff: Generative Optimization with Rich Feedback, Execution Traces, and LLMs. arXiv:2406.16218v2, 2024. URL https://arxiv.org/abs/2406.16218v2. Version read: 2406.16218v2.

[5] Paul Glasserman and Caden Lin. Assessing Look-Ahead Bias in Stock Return Predictions Generated By GPT Sentiment Analysis. arXiv:2309.17322v1, 2023. URL https://arxiv.org/abs/2309.17322v1. Version read: 2309.17322v1.

[6] Tilmann Gneiting. Making and Evaluating Point Forecasts. *Journal of the American Statistical Association*, 106(494):746–762, 2011. doi: 10.1198/jasa.2011.r10138. URL https://arxiv.org/abs/0912.0902v2. Methods read in author preprint arXiv:0912.0902v2.

[7] Saizhuo Wang, Hang Yuan, Lionel M. Ni, and Jian Guo. QuantAgent: Seeking Holy Grail in Trading by Self-Improving Large Language Model. arXiv:2402.03755v1, 2024. URL https://arxiv.org/abs/2402.03755v1. Version read: 2402.03755v1.

[8] Halbert White. A Reality Check for Data Snooping. *Econometrica*, 68(5):1097–1126, 2000. doi: 10.1111/1468-0262.00152. URL https://bashtage.github.io/kevinsheppard.com/files/teaching/mfe/advanced-econometrics/White.pdf.

[9] Yumo Xu and Shay B. Cohen. Stock Movement Prediction from Tweets and Historical Prices. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics* (*Volume 1: Long Papers*), pages 1970–1979. Association for Computational Linguistics, 2018. doi: 10.18653/v1/P18-1183. URL https://aclanthology.org/P18-1183/.

[10] Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yupeng Cao, Zhi Chen, Jordan W. Suchow, Rong Liu, Zhenyu Cui, Zhaozhuo Xu, Denghui Zhang, Koduvayur Subbalakshmi, Guojun Xiong, Yueru He, Jimin Huang, Dong Li, and Qianqian Xie. FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making. arXiv:2407.06567v3, 2024. URL https://arxiv.org/abs/2407.06567v3. Version read: 2407.06567v3.